



Shortest paths
with capacity
constraints

V. Batagelj

Motivation

Length and
capacity
semiring

Conclusions

References

Shortest paths with capacity constraints

Vladimir Batagelj

IMFM Ljubljana, UP IAM Koper, UL FMF Ljubljana

1370. sredin seminar

On Zoom, 15. oktober 2025

- 1 Motivation
- 2 Length and capacity semiring
- 3 Conclusions
- 4 References



Vladimir Batagelj: vladimir.batagelj@fmf.uni-lj.si

Last version of slides (October 15, 2025, 07 : 11): [sreda1370.pdf](#)

Shortest paths with capacity constraints

Shortest paths with capacity constraints

V. Batagelj

Motivation

Length and capacity semiring

Conclusions

References



About a month ago, I wondered what the shortest path problem is if, for individual links, in addition to the lengths, we also know the load capacities of the bridges on them.

For a vehicle with a given mass w , the problem is equivalent to the shortest path problem on the link cut at the threshold w . For the general solution of the shortest paths between all nodes, I managed to construct an appropriate semiring.

At the seminar, I will present the semiring and the development of software support for solving the shortest path problem with restrictions on the link capacities.

A **network** $\mathbf{N} = (V, L, P, W)$ consists of a **graph** $\mathbf{G} = (V, L)$, where

- V is the set of **nodes**
- $L = E \cup A$, $E \cap A = \emptyset$ is the set of **links** (A is the set of **arcs** – directed links, E is the set of **edges** – undirected links);
 $n = |V|$, $m = |L|$.
- P are node value functions / **properties**: $p : V \rightarrow R_p$; and
- W are link value functions / **weights**: $w : L \rightarrow R_w$.

Computing with link weights in networks

Shortest paths with capacity constraints

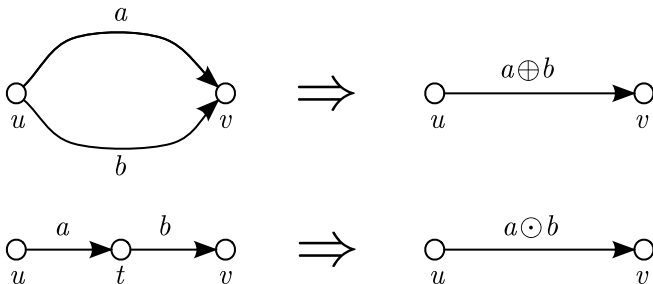
V. Batagelj

Motivation

Length and capacity semiring

Conclusions

References



A **semiring** is a "natural" algebraic structure to formalize computations with link weights in networks. They allow us to extend the weights from links to nodes, walks (paths), and sets of walks.



Semiring

Shortest paths
with capacity
constraints

V. Batagelj

Motivation

Length and
capacity
semiring

Conclusions

References

Let $\mathbb{A}$ be a set and a, b, c elements from $\mathbb{A}$. A **semiring** $[5, 3, 9, 2, 1]$ is an algebraic structure $(\mathbb{A}, \oplus, \odot, 0, 1)$ with two binary operations (addition $\oplus$ and multiplication $\odot$) where:

$(\mathbb{A}, \oplus, 0)$ is an **abelian monoid** with the neutral element 0 (zero):

$$a \oplus b = b \oplus a \quad - \text{commutativity}$$

$$(a \oplus b) \oplus c = a \oplus (b \oplus c) \quad - \text{associativity}$$

$$a \oplus 0 = a \quad - \text{existence of zero}$$

$(\mathbb{A}, \odot, 1)$ is a **monoid** with the neutral element 1 (unit):

$$(a \odot b) \odot c = a \odot (b \odot c) \quad - \text{associativity}$$

$$a \odot 1 = 1 \odot a = a \quad - \text{existence of a unit}$$

Multiplication $\odot$ **distributes** over addition $\oplus$:

$$a \odot (b \oplus c) = a \odot b \oplus a \odot c \quad (b \oplus c) \odot a = b \odot a \oplus c \odot a$$

In formulas we assume precedence of the multiplication over the addition.

A semiring $(\mathbb{A}, \oplus, \odot, 0, 1)$ is *complete* iff the addition is well defined for countable sets of elements and the commutativity, associativity, and distributivity hold in the case of countable sets.

These properties are generalized in this case; for example, the distributivity takes the form

$$\left(\bigoplus_i a_i\right) \odot \left(\bigoplus_j b_j\right) = \bigoplus_i \left(\bigoplus_j (a_i \odot b_j)\right) = \bigoplus_{i,j} (a_i \odot b_j)$$

The addition is *idempotent* iff $a \oplus a = a$ for all $a \in \mathbb{A}$. In this case the semiring over a finite set $\mathbb{A}$ is complete.

A semiring $(\mathbb{A}, \oplus, \odot, 0, 1)$ is *closed* iff for the additional (unary) *closure* operation $*$ it holds for all $a \in \mathbb{A}$:

$$a^* = 1 \oplus a \odot a^* = 1 \oplus a^* \odot a.$$

Different closures over the same semiring can exist. A complete semiring is always closed for the closure $a^* = \bigoplus_{i \in \mathbb{N}} a^i$.

In a closed semiring we can also define a *strict closure* $\bar{a}$ as $\bar{a} = a \odot a^*$.

In a semiring $(\mathbb{A}, \oplus, \odot, 0, 1)$ the *absorption* law holds iff for all $a, b, c \in \mathbb{A}$:

$$a \odot b \oplus a \odot c \odot b = a \odot b.$$

For the absorption law, because of the distributivity, it is sufficient to check the property $1 \oplus c = 1$ for all $c \in \mathbb{A}$.

Fletcher's algorithm

Shortest paths
with capacity
constraints

V. Batagelj

Motivation

Length and
capacity
semiring

Conclusions

References

Let $\mathbf{W}$ be a network weight matrix. Matrices over a semiring form a semiring for matrix addition and multiplication.

In a complete semiring, also the closure matrix $\mathbf{W}^*$ exists and can be computed using the *Fletcher's algorithm* [8, 6]

$$\mathbf{C}_0 \leftarrow \mathbf{W}$$

for $k \in 1, \dots, n$ **do**

for $i \in 1, \dots, n$ **do** **for** $j \in 1, \dots, n$ **do**

$$c_k[i, j] \leftarrow c_{k-1}[i, j] \oplus c_{k-1}[i, k] \odot c_{k-1}[k, k]^* \odot c_{k-1}[k, j]$$

$$c_k[k, k] \leftarrow 1 \oplus c_k[k, k]$$

$$\mathbf{W}^* \leftarrow \mathbf{C}_n$$

In an absorptive semiring $c_{k-1}[k, k]^* = 1$, $c_k[k, k] = 1$, and because of the addition's idempotence, we can compute in place – omit subscripts in the algorithm.

Some semirings used in network data analysis

Shortest paths
with capacity
constraints

V. Batagelj

Motivation

Length and
capacity
semiring

Conclusions

References

- 1 Combinatorial: $(\mathbb{N}, +, \cdot, 0, 1)$ or $(\mathbb{R}_0^+, +, \cdot, 0, 1)$
- 2 Reachability: $(\{0, 1\}, \vee, \wedge, 0, 1)$
- 3 Shortest paths [2]: $(\mathbb{R}_0^+, \min, +, \infty, 0)$
- 4 MaxMin (capacity): $(\mathbb{R}_0^+, \max, \min, 0, \infty)$
- 5 Pathfinder [10, 11, 12]: $(\overline{\mathbb{R}}_0^+, \min, \boxplus, \infty, 0)$
where $a \boxplus b = \sqrt[r]{a^r + b^r}$ (Minkowski)
- 6 Regular languages [13] – regular algebra (union, concatenation, and iteration of sets of strings over an alphabet A).

Length and capacity semiring

Shortest paths
with capacity
constraints

V. Batagelj

Motivation

Length and
capacity
semiring

Conclusions

References

An arc p in the weighted network (V, L, w) has weight described by a pair $w(p) = (d(p), c(p))$ where $d(p)$ is the length of the arc p and $c(p)$ is the capacity of the arc p . These weights can be extended to the paths and set of paths and described by a sequence of pairs

$$\mathbf{a} = [(ad_1, ac_1), (ad_2, ac_2), \dots, (ad_k, ac_k)]$$

where ad_i, ac_i are nonnegative real numbers and

$$i < j \Rightarrow (ac_i < ac_j) \wedge (ad_i < ad_j)$$

We will call them the *length and capacity weights* and denote their set with $\mathbf{C}$.

The interpretation of the pair (ad_i, ac_i) is: if the vehicle weight $w \in (ac_{i-1}, ac_i]$ then the shortest path has length ad_i .

Length and capacity semiring

Shortest paths
with capacity
constraints

V. Batagelj

Motivation

Length and
capacity
semiring

Conclusions

References

For a given sequence $\mathbf{a}$ and a weight w , the corresponding pair (ad_i, w) is denoted by $\mathbf{a}(w)$.

We can introduce in $\mathbf{C}$ two operations defined pointwise for $w \in \mathbb{R}_0^+ \cup \{\infty\}$

addition $(\mathbf{a} \oplus \mathbf{b})(w) = (\min(d(\mathbf{a}(w)), d(\mathbf{b}(w))), w)$ and

multiplication $(\mathbf{a} \odot \mathbf{b})(w) = (d(\mathbf{a}(w)) + d(\mathbf{b}(w)), w)$

Because the pointwise structure $(\mathbb{R}_0^+ \cup \{\infty\}, \min, +, \infty, 0)$ is an absorptive semiring so is also the structure $(\mathbf{C}, \oplus, \odot, \mathbf{0}, \mathbf{1})$, where $\mathbf{0} = [(\infty, \infty)]$ and $\mathbf{1} = [(0, \infty)]$.

$$\mathbf{1} \oplus \mathbf{a} = \mathbf{1}$$

Length and capacity semiring

Shortest paths with capacity constraints

V. Batagelj

Motivation

Length and
capacity
semiring

Conclusions

References

```
> A <- rbind( c(7,15), c(15,20), c(18,30), c(28,Inf) )
> B <- rbind( c(11,20), c(14,30), c(24,40), c(31,Inf) )
> C <- rbind( c(10,20), c(15,Inf) )
> D <- rbind( c(7,15), c(20,Inf) )
> Z <- rbind( c(Inf,Inf) )
> E <- rbind( c(0,Inf) )
```

```
> A
      [,1] [,2]
[1,]     7    15
[2,]    15    20
[3,]    18    30
[4,]    28   Inf
> B
      [,1] [,2]
[1,]    11    20
[2,]    14    30
[3,]    24    40
[4,]    31   Inf
```

Length and capacity semiring

addition and multiplication of length and capacity weights

Shortest paths with capacity constraints

V. Batagelj

Motivation

Length and capacity semiring

Conclusions

References

```
sumW <- function(A,B) {
  na = nrow(A); nb <- nrow(B)
  ia <- 1; ib <- 1; C <- NULL; do <- -1
  repeat {
    if(ia<=na) {da <- A[ia,1]; ta <- A[ia,2]} else {da <- ta <- Inf}
    if(ib<=nb) {db <- B[ib,1]; tb <- B[ib,2]} else {db <- tb <- Inf}
    d <- min(da,db); t <- min(ta,tb)
    if(d==do) C[nrow(C),2] <- t else C <- rbind(C,c(d,t))
    do <- d
    if(t>=ta) ia <- ia+1
    if(t>=tb) ib <- ib+1
    if((ia>na)&(ib>nb)) break
  }
  return(C)
}
```

```
mulW <- function(A,B) {
  na = nrow(A); nb <- nrow(B)
  ia <- 1; ib <- 1; C <- NULL; do <- -1
  while((ia<=na)&(ib<=nb)) {
    ta <- A[ia,2]; tb <- B[ib,2]
    d <- A[ia,1]+B[ib,1]; t <- min(ta,tb)
    if(d==do) C[nrow(C),2] <- t else C <- rbind(C,c(d,t))
    do <- d
    if(t>=ta) ia <- ia+1
    if(t>=tb) ib <- ib+1
  }
  return(C)
}
```

```
> sumW(A,Z)
      [,1] [,2]
[1,]    7   15
[2,]   15   20
[3,]   18   30
[4,]   28  Inf
```

```
> mulW(A,E)
      [,1] [,2]
[1,]    7   15
[2,]   15   20
[3,]   18   30
[4,]   28  Inf
```

```
> sumW(A,E)
      [,1] [,2]
[1,]    0  Inf
```

```
> mulW(A,Z)
      [,1] [,2]
[1,]  Inf  Inf
```



Length and capacity semiring

addition and multiplication of length and capacity weights

Shortest paths
with capacity
constraints

V. Batagelj

Motivation

Length and
capacity
semiring

Conclusions

References

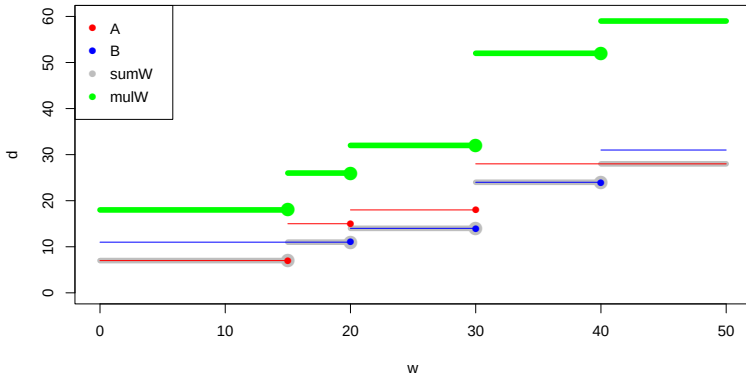
```
> S <- sumW(A,B); R <- mulW(A,B)
> A
      [,1] [,2]
[1,] 7    15
[2,] 15   20
[3,] 18   30
[4,] 28  Inf

      [,1] [,2]
[1,] 11   20
[2,] 14   30
[3,] 24   40
[4,] 31  Inf

> B
      [,1] [,2]
[1,] 7    15
[2,] 11   20
[3,] 14   30
[4,] 24   40
[5,] 28  Inf

> S
      [,1] [,2]
[1,] 7    15
[2,] 11   20
[3,] 14   30
[4,] 24   40
[5,] 28  Inf

> R
      [,1] [,2]
[1,] 18   15
[2,] 26   20
[3,] 32   30
[4,] 52   40
[5,] 59  Inf
```





Length and capacity semiring

visualization of length and capacity weights

Shortest paths
with capacity
constraints

V. Batagelj

Motivation

Length and
capacity
semiring

Conclusions

References

```
plotW <- function(A,col="black",lwd=2,pch=16,cex=1){
  na <- nrow(A)
  Ax <- rbind(c(0,0),A)
  Ax[na+1,2] <- min(Ax[na+1,2],50)
  for(i in 1:na)
    points(c(Ax[i,2],Ax[i+1,2]),rep(Ax[i+1,1],2),col=col,type="l",lwd=lwd)
  points(A[,2],A[,1],col=col,pch=pch,cex=cex)
}

> xMax <- 50; yMax <- 60;
> plot(0,0,xlim=c(0,xMax),ylim=c(0,yMax),type="n",main="sumW and mulW",
+      xlab="w",ylab="d")
> plotW(S,col="gray",lwd=6,cex=2)
> plotW(R,col="green",lwd=6,cex=2)
> plotW(A,col="red",lwd=1)
> plotW(B,col="blue",lwd=1)
> legend("topleft",c("A","B","sumW","mulW"), pch=c(16,16,16,16),
+       col=c("red","blue","gray","green"))
```

Drawing a network (see next slide)

```
> lab <- gsub(" ","",gsub("Inf",intToUtf8(8734),gsub("c","",as.character(E(N)$cw))))
> lab[(N$mo+1):length(lab)] <- ""
> plot(N,vertex.size=20,vertex.label.cex=2,edge.label=lab,edge.label.cex=1.5)
```


Length and capacity semiring

example network

Shortest paths with capacity constraints

V. Batagelj

Motivation

Length and capacity semiring

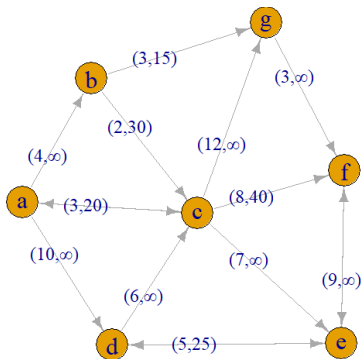
Conclusions

References

```
> setwd("C:/Users/vlado/work/R/semi")
> source("capacity.R")
> library(igraph); library(data.table)
> N <- readRDS("semit2.rds")
> n <- gorder(N); m <- gsize(N)
> nodes <- as_data_frame(N, what="vertices")
> links <- as_data_frame(N, what="edges")
```

```
> nodes
  name    x    y
a     70  210
b    184  391
c    358  194
d    218    0
e    594    3
f    599  256
g    471  473

> links
  from to weight    cw dir
1    a  b     4, Inf  a
2    a  c     3, 20  e
3    a  d    10, Inf  a
4    a  c     2, 30  a
5    b  c     6, Inf  a
6    b  d     3, 15  a
7    b  e     5, 25  e
8    c  d    12, Inf  a
9    c  e     7, Inf  a
10   c  f     8, 40  e
11   c  f     9, Inf  e
12   c  a     3, Inf  e
13   c  a     3, 20  e
14   e  d     5, 25  e
15   f  e     9, Inf  e
```



```
> sapply(graph_attr_names(N), function(x) graph_attr(N, x))
```

```
> setwd("C:/Users/vlado/work/R/semi")
> source("capacity.R")
> library(igraph); library(data.table)
> CW <- function(p) rbind(C$cw[p][[1]])
> N <- readRDS("semiT2.rds")
> n <- gorder(N); nodes <- as_data_frame(N, what="vertices")
> Z <- rbind(c(Inf, Inf)); E <- rbind(c(0, Inf))
> # initialize closure matrix
> L <- CJ(nodes$name, nodes$name); ZZ <- vector("list", n*n)
> for(i in 1:n**2) ZZ[[i]] <- Z
> C <- data.frame(from=L$V1, to=L$V2); C$cw <- ZZ
> for(p in 1:m){ uv <- as.vector(ends(N, p, names=FALSE))
+   C$cw[(uv[1]-1)*n + uv[2]] <- E(N)$cw[p]
+ }
> # Fletcher's algorithm
> for(t in 1:n){
+   for(u in 1:n) for(v in 1:n) { uv <- (u-1)*n + v
+     ut <- (u-1)*n + t; tv <- (t-1)*n + v
+     C$cw[uv][[1]] <- sumW(CW(uv), mulW(CW(ut), CW(tv)))
+   }
+   tt <- (t-1)*n + t; C$cw[tt][[1]] <- sumW(E, CW(tt))
+ }
```



Length and capacity semiring

Closure

Shortest paths
with capacity
constraints

V. Batagelj

Motivation

Length and
capacity
semiring

Conclusions

References

```
> C
  from to
1     a  a      0, Inf
2     a  b      4, Inf
3     a  c      3, 6, 16, 20, 30, Inf
4     a  d      10, 13, 23, 20, 30, Inf
5     a  e      10, 11, 14, 24, 31, 15, 20, 30, 40, Inf
6     a  f      7, 15, 18, 28, 15, 20, 30, Inf
7     a  g      5, Inf, 20, Inf
8     b  a      2, Inf, 30, Inf
9     b  b      14, Inf, 25, Inf
10    b  c      9, Inf, 30, Inf
11    b  d      6, 10, Inf, 15, 30, Inf
12    b  e      3, 14, Inf, 15, 30, Inf
13    b  f      3, Inf, 20, Inf
14    b  g      7, Inf, 20, Inf
15    c  a      0, Inf
16    c  b      12, Inf, 25, Inf
17    c  c      7, Inf
18    c  d      8, 15, 40, Inf
19    c  e      10, 12, 15, Inf
20    c  f      9, Inf, 20, Inf
21    d  a
22    ...
34    e  f      9, Inf
35    e  g      21, 23, Inf, 15, 25, Inf
36    f  a      23, Inf, 20, Inf
37    f  b      27, Inf, 20, Inf
38    f  c      20, Inf, 25, Inf
39    f  d      14, Inf, 25, Inf
40    f  e      9, Inf
41    f  f      0, Inf
42    f  g      30, 32, Inf, 15, 25, Inf
43    g  a      26, Inf, 20, Inf
44    g  b      30, Inf, 20, Inf
45    g  c      23, Inf, 25, Inf
46    g  d      17, Inf, 25, Inf
47    g  e      12, Inf
48    g  f      3, Inf
49    g  g      0, Inf
```

```
> C$cw[6]
[[1]]
[,1] [,2]
[1,] 10 15
[2,] 11 20
[3,] 14 30
[4,] 24 40
[5,] 31 Inf
> C$cw[46]
[[1]]
[,1] [,2]
[1,] 17 25
[2,] Inf Inf
```

Length and capacity semiring

Extended closure

Shortest paths
with capacity
constraints

V. Batagelj

Motivation

Length and
capacity
semiring

Conclusions

References

To be able to reconstruct also the minimal paths, we extend the pairs (d, c) to triples (d, c, t) - the pair (d, c) is realized by passing through the node t . Node 0 indicates a direct link.

Extended operations

```
sumT <- function(A,B){
  na = nrow(A); nb <- nrow(B)
  ia <- 1; ib <- 1; C <- NULL; do <- -1
  repeat {
    if(ia<=na) {da <- A[ia,1]; ta <- A[ia,2]; ka <- A[ia,3]} else {da <- ta <- Inf}
    if(ib<=nb) {db <- B[ib,1]; tb <- B[ib,2]; kb <- B[ib,3]} else {db <- tb <- Inf}
    if(da<db) {d <- da; k <- ka} else {d <- db; k <- kb}
    t <- min(ta,tb)
    if(d==do) {C[nrow(C),2] <- t; C[nrow(C),3] <- k} else C <- rbind(C,c(d,t,k))
    do <- d
    if(t>=ta) ia <- ia+1
    if(t>=tb) ib <- ib+1
    if((ia>na)&(ib>nb)) break
  }
  return(C)
}

mult <- function(A,B,k){
  na = nrow(A); nb <- nrow(B)
  ia <- 1; ib <- 1; C <- NULL; do <- -1
  while((ia<=na)&(ib<=nb)) {
    ta <- A[ia,2]; tb <- B[ib,2]
    d <- A[ia,1]+B[ib,1]; t <- min(ta,tb)
    if(d==do) C[nrow(C),2] <- t else C <- rbind(C,c(d,t,k))
    do <- d
    if(t>=ta) ia <- ia+1
    if(t>=tb) ib <- ib+1
  }
  return(C)
}
```



Length and capacity semiring

Extended closure

Shortest paths
with capacity
constraints

V. Batagelj

Motivation

Length and
capacity
semiring

Conclusions

References

```

closureT <- function(N){
  n <- gorder(N); m <- gsize(N)
  nodes <- as_data_frame(N,what="vertices")
  Z <- rbind(C(Inf,Inf)); E <- rbind(c(0,Inf))
  # initialize closure matrix
  L <- CJ(nodes$name,nodes$name)
  ZZ <- vector("list",n**2)
  for(i in 1:n**2) ZZ[[i]] <- cbind(Z,0)
  C <- data.frame(from=L$V1,to=L$V2); C$cw <- ZZ
  for(p in 1:m){ uv <- as.vector(ends(N,p,names=FALSE))
    C$cw[(uv[1]-1)*n + uv[2]] <- list(cbind(rbind(E(N)$cw[p][[1]]),0))
  }
  # Fletcher's algorithm
  for(t in 1:n){
    for(u in 1:n) for(v in 1:n) { uv <- (u-1)*n + v
      ut <- (u-1)*n + t; tv <- (t-1)*n + v
      C$cw[uv][[1]] <- sumT(CW(C,uv),mulT(CW(C,ut),CW(C,tv),t))
    }
    tt <- (t-1)*n + t; C$cw[tt][[1]] <- sumT(cbind(E,0),CW(C,tt))
  }
  return(C)
}

> N <- readRDS("semit2.rds")
> n <- gorder(N)
> CX <- closureT(N)
> nodes <- as_data_frame(N,what="vertices")
> CNx <- graph_from_data_frame(CX,directed=TRUE,vertices=nodes)
> CNx$name <- "semit2 extended closure"
> CNx$tit <- "Extended closure of the Test network 2 for the capacity semiring"
> CNx$by <- "Vladimir Batagelj"
> CNx$date <- date()
> saveRDS(CNx,file="xclosureT2.rds")
> write_graph_netsJSON(CNx,file="xclosureT2.json")

```



Length and capacity semiring

Extended closure

Shortest paths
with capacity
constraints

V. Batagelj

Motivation

Length and
capacity
semiring

Conclusions

References

```
> CX
      from to
1      a  a      0, Inf, 0
2      a  b      4, Inf, 0
3      a  c      3, 6, 16, 20, 30, Inf, 0, 2, 4
4      a  d      10, Inf, 0
5      a  e      10, 13, 23, 20, 30, Inf, 3, 3, 4
6      a  f      10, 11, 14, 24, 31, 15, 20, 30, 40, Inf, 7, 3, 3, 4, 7
7      a  g      7, 15, 18, 28, 15, 20, 30, Inf, 2, 3, 3, 4
8      b  a      5, Inf, 20, Inf, 3, 7
20.... c  f      8, 15, 40, Inf, 0, 7
24.... d  c      6, Inf, 0
27.... d  f      14, 21, 40, Inf, 3, 7
42.... f  g      30, 32, Inf, 15, 25, Inf, 5, 5, 7
48.... g  f      3, Inf, 0
49.... g  g      0, Inf, 0
> rbind(V(N)$name)
      [,1] [,2] [,3] [,4] [,5] [,6] [,7]
[1,] "a"  "b"  "c"  "d"  "e"  "f"  "g"
> CW(CX,6)
      [,1] [,2] [,3]
[1,] 10    15    7
[2,] 11    20    3
[3,] 14    30    3
[4,] 24    40    4
[5,] 31    Inf    7
> # w=40 1/a-4/d-6/f; a-d > 4 link; d-f > 27
> CW(CX,27)
      [,1] [,2] [,3]
[1,] 14    40    3
[2,] 21    Inf    7
> # w=40 4/d-3/c-6/f; d-c > 24 link; c-f > 20
> CW(CX,20)
      [,1] [,2] [,3]
[1,] 8     40    0
[2,] 15    Inf    7
> # 1/a-4/d-3/c-6/f -> 10+6+8 = 24
```



Length and capacity semiring

Path reconstruction

Shortest paths with capacity constraints

V. Batagelj

Motivation

Length and capacity semiring

Conclusions

References

```
CW <- function(C,p) rbind(C$cw[p][[1]])

nodex <- function(C,u,v,c){
  uv <- (u-1)*n + v; T <- CW(C,uv)
  for(i in 1:nrow(T)) if(T[i,2]>=c) break
  return(T[i,3])
}

.path <- function(C,u,v,c){
  k <- nodex(C,u,v,c)
  if(k == 0) return(NULL)
  return(c(.path(C,u,k,c), k, .path(C,k,v,c)))
}

path <- function(C,u,v,c){
  if(nodex(C,u,v,c) == Inf) return(NULL)
  return(c(u, .path(C,u,v,c), v))
}

paths <- function(C,u,v){
  uv <- (u-1)*n + v; T <- CW(C,uv)
  P <- data.frame(d=T[,1],c=T[,2]); P$P <- vector("list",nrow(T))
  for(i in 1:nrow(T)) P$P[i] <- list(path(C,u,v,T[i,2]))
  return(P)
}

> (P <- paths(CX,1,6))
   d   c   1, 2, 7, 6
1 10  15
2 11  20
3 14  30
4 24  40
5 31 Inf 1, 4, 3, 7, 6
> Q <- P
> for(i in 1:nrow(P)) Q$P[i] <- list(nodes$name[P$P[i][[1]]])
> Q
   d   c   a, b, g, f
1 10  15
2 11  20
3 14  30
4 24  40
5 31 Inf a, d, c, g, f
```



Conclusions

Shortest paths
with capacity
constraints

V. Batagelj

Motivation

Length and
capacity
semiring

Conclusions

References

- Instead of the weight capacity, other constraints can be used (height, width, speed, etc.). They can also be combined.
- I am still looking for some real-life length and capacity networks.
- Recently, a faster algorithm for solving the single-source shortest path problem was published [7, 4].



Acknowledgments

Shortest paths
with capacity
constraints

V. Batagelj

Motivation

Length and
capacity
semiring

Conclusions

References

The computational work reported in this presentation was performed using **R**. The code and data are available at [GitHub/bavla/semirings](https://github.com/bavla/semirings).

This work was partially supported by the Slovenian Research Agency (research program P1-0294 and research project J5-4596) and was prepared within the framework of COST action CA21163 (HiTEc).

-  Abdali SK, Saunders BD (1985) Transitive closure and related semiring properties via eliminants. Theor Comput Sci 40:257—274
-  Baras JS, Theodorakopoulos G (2010) Path problems in networks. Morgan & Claypool, Berkeley
-  Batagelj V (1994) Semirings for social networks analysis. J Math Soc 19(1):53—68
-  Brubaker B (2025) New Method Is the Fastest Way To Find the Best Routes. Quanta magazine, August 6, 2025. [WWW](https://www.quantamagazine.org/a-new-method-for-finding-the-fastest-way-to-find-the-best-routes-20250806/)
-  Carré B (1979) Graphs and networks. Clarendon, Oxford
-  Cerinšek M, Batagelj V (2017) Semirings and Matrix Analysis of Networks. In: Alhajj R., Rokne J. (eds) Encyclopedia of Social Network Analysis and Mining. Springer, New York, NY.



Duan R, Mao J, Mao X, Shu X, Yin L (2025) Breaking the Sorting Barrier for Directed Single-Source Shortest Paths. April 23, 2025.

[arXiv](#)



Fletcher JG (1980) A more general algorithm for computing closed semiring costs between vertices of a directed graph. Commun ACM 23(6):350–351



Gondran M, Minoux M (2008) Graphs, dioids and semirings: new models and algorithms. Springer, New York



Schvaneveldt RW, Dearholt DW, Durso FT (1988) Graph theoretic foundations of Pathfinder networks. Comput Math Appl 15(4):337–345



Quirin A, Córdón O, Santamaria J, Vargas-Quesada B, Moya-Anegón F (2008) A new variant of the Pathfinder algorithm to generate large visual science maps in cubic time. Inf Process Manag 44(4): 1611–1623



Vavpetič A, Batagelj V, Podpečan V (2009) An implementation of the Pathfinder algorithm for sparse networks and its application on text networks. Proceedings of the 12th international multiconference Information Society, Volume A, 236–239. [PDF](#)



Wikipedia: [Regular language](#).